

Identifying Public Data in Constant-Time Programs with Symbolic Execution

Yavor Litchev
ylitchev@stanford.edu
Stanford University
Stanford, CA, USA

Jane Nguyen
janen9@stanford.edu
Stanford University
Stanford, CA, USA

ACM Reference Format:

Yavor Litchev and Jane Nguyen. 2026. Identifying Public Data in Constant-Time Programs with Symbolic Execution. In . ACM, New York, NY, USA, 7 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Modern processors leverage optimizations including branch prediction and speculative execution to reap performance improvements. By guessing the branch destination and speculatively executing subsequent instructions, the CPU allows instructions to enter the pipeline before the branch decision is determined. Speculative instructions can access and modify memory and registers, but the instructions are not committed if the prediction is ultimately determined to be false. However, in speculatively executing instructions that will ultimately be discarded, the CPU can leak victim "secrets", to adversaries through microarchitectural side channels in Spectre attacks. [8] This is especially critical for evaluation in cryptographic and security-critical code, and appropriate defenses must be investigated to uphold security guarantees.

Speculative execution attacks consist of a secret value speculatively accessed and read into architectural state and a transmitter, any instruction whose execution creates operand-dependent hardware resource usage [15]. These attacks have motivated prior work to mitigate transmitter leakage. Constant-time (CT) programming paradigms dictate that secrets must not influence observable behavior. An implementation's logical execution time should be independent of secrets and follow strict guidelines to avoid variable-time operations, control flow, and memory access patterns reliant on secret data [2]. However, CT or data oblivious programs are not guaranteed to block side channel leakage with existing ISAs,

as hardware can apply invisible optimizations that undermine software transformations [14]. CT programs are thus tricky to implement and insufficient for security guarantees.

To protect against speculative execution attacks, which undermine CT programming, prior work has introduced the notion of public vs. private [4]. "Public" denotes observable data, whereas "private" denotes secret data that needs protection. Spectector formally defines a security policy as a finite subset of public register identifiers and memory addresses that are parts of the configuration known by an adversary [7]. If a register contains public data, it can be allowed to transiently leak without protection. This concept has been utilized in several papers to implement selective protections. A naive approach is to protect all registers, marking all data as private. However, this incurs high performance overhead, and as such, previous work has investigated techniques that improve performance by only protecting potentially secret data. These approaches (SPT, Declassiflow, Protean) vary in how they determine publicness and enforce selective protections through statically or dynamically tracking program execution, marking data as public or private, and enabling protections either at the hardware level, software level, or through HW-SW co-design [5] [4][11].

This paper posits an approach that is complementary to prior work, which defined publicness based on non-speculative leakage or enforced protections at the hardware or ISA-level [4][5][11]. Rather than directly modifying the program or underlying hardware, the proposed CT-Publicness tool statically analyzes the program at the Intermediate Representation (IR) level, determining publicness at each program point in a given function through dual symbolic execution of control-flow paths with a transmitter-based leakage model. A value is public along a given path if it can be assigned different values between instances, indicating its value is not solely a function of architecturally transmitted data; a value is public at a program point if it is public along all paths through that point. Our contributions are as follows:

- CT-Publicness: A complete toolchain for static analysis of public vs. potentially secret at program points, consisting of an LLVM IR Pass for trace generation and a Publicness Inference pipeline for public-at-program-point determination.
- An evaluation of our approach against naive baselines, demonstrating nontrivial reductions in overprotection.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. *Conference'17, Washington, DC, USA*

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2026/03

<https://doi.org/XXXXXXX.XXXXXXX>

2 Background

Timing and cache side channels have been known to break cryptographic implementations. Secret-dependent control flow and secret-dependent data flow can result in different observable behaviors[10]. Kocher's timing attacks show how execution-time variation in private-key operations can reveal secret material, and Bernstein demonstrated complete AES key recovery via cache-timing leakage from table-driven implementations [9][3]. Such attacks have motivated constant-time coding paradigms and further defenses against speculative execution attacks that undermine these paradigms.

2.1 Constant-Time Programming Paradigms

CT programming involves carefully writing programs to avoid side channels by only passing secret data to safe instructions, in which execution creates *operand-independent* hardware resource usage. CT programming ensures the program does not have private data-dependent branches or unsafe instructions with private data-dependent operands. What constitutes a safe vs. unsafe instruction, however, is microarchitecture-dependent, making these paradigms difficult to uphold across various microarchitectures [6]. With modern speculative processors, maintaining security through CT is even further compromised. CT programming is effective under *sequential* execution of programs, but unconstrained control flow through speculative execution risks leakage of secrets to transmitters.

2.2 Transmitter Leakage

Unsafe instructions are information *transmitters*, encompassing control-flow, memory, and variable-time instructions. Even with CT protections, transient or speculative execution can steer secrets toward transmitter operands. Modern processors exploit instruction-level parallelism (ILP) through branch prediction: conditional branch, indirect branch, and return address prediction. [12] This speculative branching can transmit secrets since the branch decision can affect observable microarchitectural state differently based on the execution path chosen. For example, a memory access can occur, loading a specific cache line based on speculation. Even if this instruction is later squashed, the cache state remains, revealing secrets if the attacker probes this cache state. Store-to-load forwarding and predictive store forwarding are further vulnerable memory-based optimizations. Variable-time instructions leak information through timing observations. Floating point, division, and modulo instructions are some notable sources of variable-time operand-dependent behavior. Figure 1 illustrates a few of these transmitters.

2.3 Speculative Execution Defenses

A naive approach is to protect all registers and all memory accesses, but this approach is far too expensive, undermining the purpose of optimizations like speculation to begin

```
int foo(int *p, int secret, int x) {
    int y = x + 1;
    int z = *p;           // load address transmitter
    if (secret & 1) {      // branch condition transmitter
        y = y + z;
    }
    *p = y;               // store address transmitter
    return y;
}
```

Figure 1. Example Function with Transmitters

with. As a result, substantial prior work has investigated defenses for speculative execution, but some approaches still overprotect or require additional hardware overhead. Strict defenses such as NDA restrict speculative data execution with variants for different attacks and security/performance tradeoffs [13]. Speculative Load Hardening (SLH), protects against leaks by tracking speculation state, and masking values during misspeculation. It introduces notable overhead through additional instructions and bitwise operations to enforce the speculation-dependent masks created [16].

Recent work attempts to reduce the nontrivial overheads associated with speculative execution defenses. Speculative Privacy Tracking (SPT) utilizes hardware to dynamically track execution, marking data as public if leaked at instruction retirement [5]. Declassiflow statically removes protection from points in the program already guaranteed to leak non-speculatively [4]. Protean introduces an ISA extension and interface to communicate from software to hardware which registers and memory require protection as well as a compiler to infer protections and hardware mechanisms to enforce these software-defined protections [11].

2.4 Motivation

Existing constant-time paradigms prevent certain classes of architectural leakage, but they do not fully account for speculative microarchitectural leakage. Prior work explores speculative execution defenses at different abstraction layers, from inserting instructions at software-level to introducing ISA extensions or adding hardware overhead. The notion of publicness is also often defined based on information revealed by non-speculative execution. Thus, deeper investigation into particular abstraction layers as well as other formal definitions of publicness can complement existing defenses. This motivates this paper's analysis at the granularity of *intermediate values* and a definition of publicness directly related to the *symbolic* nature of a program. This static analysis enables the compiler or downstream defenses to make precise protection decisions based upon this analysis. Put succinctly, our problem statement is as follows: Given an LLVM Intermediate Representation (IR) and a transmitter-based leakage model, determine which registers are public at each program point in a given function.

3 Approach

We define and compute publicness statically at the level of LLVM intermediate values, leveraging dual symbolic execution and SMT solving to determine whether a value is public or potentially private. If a value is provably public, then leaving it unprotected will not weaken the intended security policy.

Our approach consists of two main components: Path Enumeration and Publicness Determination. We enumerate all paths through a program function, determine per-path publicness per program point, and aggregate these results across paths for program-point determination of publicness.

3.1 Path Enumeration

A program can be defined as a control flow graph (CFG): a directed graph in which the nodes represent basic blocks (BBs) and the edges represent control flow paths. A BB is a linear sequence of instructions with one entry point and one exit point [1]. The CFG denotes all possible paths through a program, and as such, these concepts enable path enumeration. We use this CFG representation to represent each program function and extract all its paths through this CFG.

In Figure 2, we illustrate the CFG for foo from Figure 1. The function is decomposed into 3 BBs. The branch condition results in 2 paths through the CFG. One path goes through the true branch based on the branch condition, resulting in a path from BB0 to BB1 to BB2. The other path takes the false branch, skipping directly from BB0 to BB2.

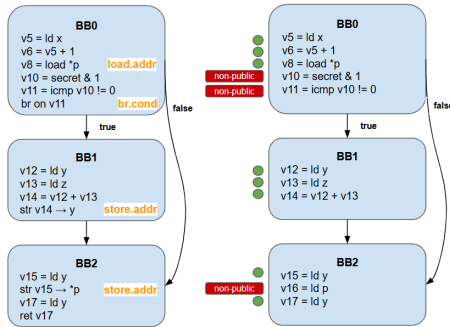


Figure 2. CFG and Publicness Determination of Figure 1 Function

3.2 Publicness Determination

Our definition of publicness is based on dual symbolic execution and a transmitter-based leakage model. We define transmitters as mentioned in Section 2.2, and include a comprehensive set of transmitters in our leakage model. We model memory instruction transmitters including load address and store address, control-flow transmitters including branch condition, switch condition, indirect branch target, and call target, and variable-time instruction transmitters

including division operand and remainder operand. We also model atomic instruction transmitters including atomic read-modify-write address and compare and exchange address. Based on the transmitter leakage model and our enumerated paths from Section 3.1, we define publicness with the following equations:

$$\text{path_public}(P, v) := \text{SAT}(\text{PC}_A(P) \wedge \text{PC}_B(P) \wedge \text{tx_equality}(P, v_A, v_B) \wedge (v_A \neq v_B)) \quad (1)$$

$$\text{public at point}(pp, v) := \forall \text{ paths } P \text{ through } pp : \text{path_public}(P, v) \quad (2)$$

For each path through the CFG, we symbolically execute the same path twice. We constrain the two executions so that every transmitter observation is equal across both executions. Under those equal-transmitter constraints and path constraints, we ask whether a given value can still differ between the two executions by passing Equation 1 to the SMT solver to determine satisfiability. If the query is satisfiable, then that value is public along that path, because transmitter observations do not determine it. If it cannot, then it is not public along that path. Finally, we aggregate across all paths that pass through a program point using Equation 2. A value is public at a program point if and only if it is public on every path through that point.

In Figure 2, the publicness determination is shown on the right. Green indicates that a value can vary even when all transmitter observations are held equal, so that value is public along the path. Red indicates that either the path constraints or the transmitter equalities force the value to be the same (non-public). Intuitively, values closely tied to secret-controlled observations remain non-public, while values that are independent of those observations remain public. Aggregating the per-path public vs. non-public results across all paths results in the final determination of public vs. potentially secret returned to the user for each program point.

4 Implementation

We implement our approach as an end-to-end analysis pipeline with two components: an LLVM-based frontend in C that extracts a finite symbolic problem from programs, and a Python backend that performs symbolic reasoning and aggregates the resulting publicness judgements. The LLVM frontend uses compiler IR, control-flow, and operand-level instruction information to identify instructions and operands relevant to the leakage model and emit machine-readable artifacts that describe instruction traces, CFGs, path conditions, and path coverage. The backend provides constraint construction, symbolic execution, and solver integration. It reconstructs path-local symbolic executions from the artifacts, executes each path twice under equal-transmitter constraints, queries an SMT solver to determine whether values may differ, and

aggregates per-path results into public-at-program-point judgments. The tool takes programs or LLVM IR, recovers relevant control-flow and leakage structure, and computes publicness judgments for SSA values at program points.

4.1 LLVM IR Pass

The frontend is implemented as an LLVM IR pass that walks each function, BB, and instruction. Its primary responsibility is to convert LLVM IR into a structured representation suitable for downstream symbolic reasoning. The pass assigns stable program-point and value identifiers, identifies transmitter operands according to our leakage model, and emits path and control-flow metadata. For each instruction, the pass produces a trace record containing the function name, and consists of basic block, program point, opcode, defined value, used values, and any transmitter annotations associated with that instruction. Stable identifiers allows the backend to later relate symbolic values, path-local definitions, and aggregation results across multiple emitted files. These trace records are written as NDJSON objects.

The pass marks transmitters across several classes of observable operands as discussed in Section 3.2 for usage by the backend to reason about transmitter equality: `load.addr`, `store.addr`, `br.cond`, `switch.cond`, `indirectbr.target`, `call.target`, `atomicrmw.addr`, `cmpxchg.addr`, `div.operand`, and `rem.operand`. The pass also emits control-flow and path artifacts. Each function is represented as a CFG. We perform DFS-based path enumeration over the CFG to recover a finite set of candidate execution paths. For each path, the pass records a path identifier, sequence of traversed BBs, and path conditions induced by branch decisions. Metadata includes mappings of program points to paths that pass through them and path-to-program-point sequences to simplify downstream aggregation.

Because unrestricted path enumeration is intractable with loops and deeply branching code, the pass uses configurable budgets on path exploration depth and total emitted paths. It also supports bounded loop revisitation, so that the analysis can reason about a finite prefix of looping behavior without attempting to enumerate infinitely many traces. It also performs basic pruning when branch or switch conditions are statically constant, removing edges known to be infeasible.

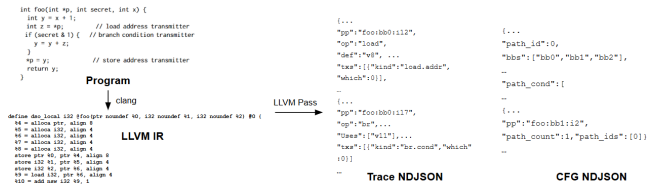


Figure 3. LLVM IR Pass

4.2 Publicness Inference Pipeline

The Python backend is responsible for turning the frontend's emitted artifacts into publicness judgments with formal logical reasoning. It consists of four phases: parsing and path reconstruction, dual symbolic execution, SMT-based publicness checking, and aggregation to program-point results.

In the first phase, the backend parses the trace and CFG artifacts and reconstructs path-specific instruction streams. It joins control-flow info with the instruction-level trace so that each path is represented not just as a sequence of BBs, but as an executable symbolic object with precise instruction order and associated path constraints. The parser also recovers transmitter metadata and program-point coverage info for later use in publicness queries and aggregation.

The tool then performs dual symbolic execution. For each enumerated path, we symbolically execute the same path twice, producing execution states A and B. Each state maintains an environment mapping SSA definitions to symbolic expressions, together with a symbolic memory model used for loads and stores. The path constraints for the chosen path are asserted in both executions. We then add equal-transmitter constraints requiring that every transmitter observation on the path be equal between executions A and B; these constraints model two executions that are indistinguishable to an adversary under the leakage model. Once the dual execution state has been constructed, the backend checks publicness by issuing SMT queries to Z3. For each relevant SSA value on the path, we ask whether the value in execution A can differ from the corresponding value in execution B while preserving path constraints and transmitter equality. If the formula is satisfiable, then the value is public along that path. If it is unsatisfiable, then the value is not public along that path. This per-path judgment is the direct implementation of the formal rule from Section 3. Finally, we aggregate these path-level results into point-level publicness. Since a program point may be reachable by multiple paths, we group path judgments by function, program point, and value identifier. A value is considered public at a program point only if it is public along every path through that point.

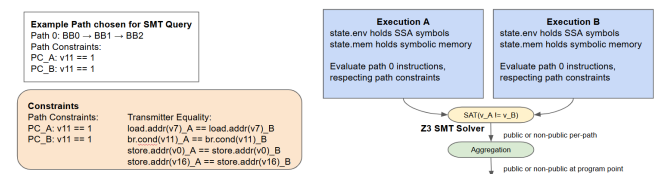


Figure 4. Publicness Inference

A notable implementation challenge was loop handling. Because our path space is bounded, repeated loop traversals reuse the same static SSA identities from the IR, which can introduce contradictions if naively interpreted as unbounded

dynamic iterations. To address this, we implemented a first-iteration loop heuristic. When the frontend emits bounded repeated paths, the backend analyzes the first loop iteration separately and uses that info to lift publicness facts to later loop-local program points. The backend also includes practical features such as recording per-function and per-path metrics including solver query counts, satisfiable vs. unsatisfiable checks, and cumulative solver time.

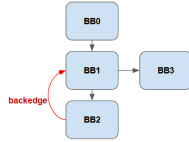


Figure 5. Loop Handling

5 Methodology

Our evaluation is designed to determine whether the tool can identify nontrivial opportunities to reduce overprotection relative to a naive baseline that treats all values as private and investigates the computational cost of running the analysis across a diverse set of benchmark functions.

Our benchmark suite consists of 39 functions across 9 source files. The suite covers a range of control-flow and semantic patterns relevant to our problem setting. It includes small sanity-check examples to validate expected behavior of the analysis on straight-line and branching code, control-flow-oriented benchmarks exercising branches, loops, and recursion, and larger kernels inspired by classical algorithms and cryptographic computations. Representative examples include toy kernels with easily understood publicness structure, classical workloads such as quicksort partition and traveling-salesman-style routines, and crypto-style kernels such as Paillier-inspired lookups, square-and-multiply code, and compression-round computations. The evaluation pipeline follows the same structure as the implementation. Each benchmark is compiled to or provided as LLVM IR. The toolchain results in judgments as public-at-program-point outputs. Scripts collect metrics into CSVs for analysis.

Our primary evaluation baseline is a naive “100% private” policy in which every value is conservatively treated as requiring protection and no overprotection is removed. In contrast, our analysis identifies program-point/value pairs that are provably public under the transmitter-based leakage model and serve as opportunities to reduce unnecessary protections. We record several quantitative metrics with the tool. On the frontend, we measure the number of explored paths and transmitters identified. On the backend, we measure the number of SMT queries issued, satisfiable checks corresponding to public values, unsatisfiable checks corresponding to non-public values, any unknown outcomes returned by the solver, and the cumulative solver time per function. These

metrics allow us to separate structural complexity from logical complexity: some functions are difficult because they generate many paths, while others are difficult because even few paths induce solver-heavy symbolic expressions. In addition to these quantitative measurements, we use qualitative validation on examples with behavior known by construction. Several toy and microbenchmark cases were designed so that the expected publicness classification is apparent by inspection for a sanity check. While this is not a full formal validation of the implementation, it demonstrates the tool makes sensible distinctions between public and non-public intermediate values.

The methodology is aimed at evaluating the tool on a representative and varied benchmark suite, quantifying both its ability to expose publicness structure and the computational effort required to do so. This aligns the evaluation with the project’s actual contribution: a practical LLVM-to-SMT pipeline for value-level publicness analysis under a transmitter-based leakage model.

6 Results

Our evaluation shows the analysis is able to recover non-trivial publicness structure across a diverse suite, identifying meaningful opportunities to reduce overprotection relative to a naive baseline (100% private). Across the full suite, the tool analyzed 39 functions from 9 source files, enumerated 63 paths, identified 609 transmitters, and issued 1388 SMT queries. These results indicate the system is capable of scaling beyond toy examples to a mixture of branching, looping, recursive, classical algorithmic, and crypto-inspired kernels.

At a qualitative level, the analysis produces a benchmark-dependent mix of public and non-public values, reflecting the interaction between dataflow, control flow, and the transmitter-based leakage model. In sanity-check examples where expected publicness structure is clear by inspection, the tool behaves as intended: values that remain independent of transmitter observations are classified as public, while values tied to observable operands are non-public. Results also show that publicness is highly nonuniform even within a single function. Some intermediate values remain public despite occurring in security-relevant code, while others become constrained by transmitter equality and thus are treated as potentially secret. This is the structure that a coarse protection policy would miss and demonstrates our tool’s utility in exposing fine-grained distinctions to enable selective mitigation. A downstream defense consuming these judgments would be able to avoid protecting certain values or operations that a blanket defense would conservatively harden.

The benchmark suite also illustrates that different classes of programs induce different publicness profiles. Straight-line arithmetic kernels and simple memory examples often preserve substantial public structure, especially when operands do not flow into modeled transmitters. In contrast,

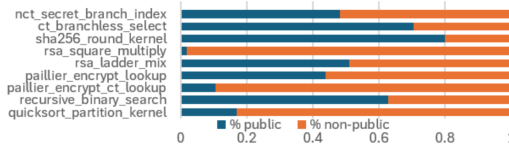


Figure 6. Testbench Public vs. Non-public

secret-dependent branching examples and certain crypto-style kernels retain larger non-public regions, since many intermediate values are linked to branch conditions, address computations, or other observable effects. Loop-oriented and recursive examples fall somewhere in between: the bounded symbolic analysis often identifies useful publicness information, but the precision of those results depends on the quality of path coverage and the effectiveness of the loop heuristic.

In addition to effectiveness, the results provide insight into the computational cost. Analysis cost depends not only on control-flow complexity, but also on semantic complexity of the symbolic constraints. Functions with multiple paths naturally increase the number of solver queries and the reasoning required. However, even functions with relatively low path counts can become expensive when they contain rich arithmetic structure, bitvector-heavy computations, or symbolic relationships difficult for the SMT solver to discharge. As a result, path count alone is not a sufficient predictor of analysis cost. This effect is visible in larger algorithmic and cryptographic kernels. Functions such as *rsa_ladder_mix* and *sha256_round_kernel* were more computationally expensive but not dominated by path explosion. Their cost reflects the complexity of the generated formulas and the number of value publicness checks required within each path. Thus, optimizing the analysis requires attention not only to path pruning and loop treatment but also to the shape and simplification of the symbolic expressions sent to the solver.

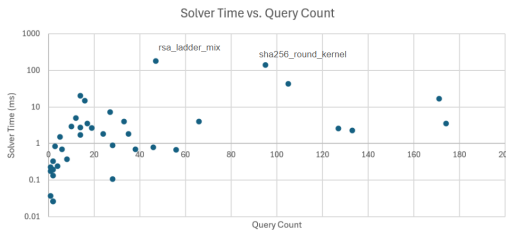


Figure 7. Solver Time (ms) vs. Query Count

These results indicate the tool succeeds at its primary goal of identifying value-level publicness distinctions that a naive all-private baseline would overlook. The current implementation is also expressive enough to analyze a varied suite, but its cost profile is shaped by both structural and semantic factors. This reinforces LLVM-level publicness analysis is feasible and informative while highlighting the challenges that must be addressed to make it robust at larger scale.

7 Discussion

The results suggest that value-level publicness analysis is a promising complement to existing speculative-execution and CT defenses by computing a finer-grained classification of which SSA values remain public at which program points under a transmitter-based leakage model. Many existing defenses pay overhead by conservatively protecting broad regions of code or large sets of values, even when some of those values do not require protection under the leakage model. Our analysis shows that this conservatism is often unnecessary; even in functions that manipulate secrets, there may still exist substantial internal structure that is public. The tool’s main contribution is that it exposes a useful intermediate representation of security-relevant information that can guide future compiler or hardware-assisted mitigations.

7.1 Limitations

A limitation is that the leakage model is transmitter-based. The analysis computes publicness relative to this transmitter set, not all conceivable leakage mechanisms. If a leakage source falls outside that model, the resulting classifications may be incomplete against a stronger adversary. Second, path coverage is finite and bounded. The tool restricts exploration using path, depth, and loop budgets for tractability, so the analysis does not reason about arbitrarily many dynamic executions or unbounded control-flow behavior and is not a fully inductive proof. Third, the symbolic executor uses an approximate memory model and only partial instruction coverage. Symbolic memory is not yet a complete SSA-memory or array-theoretic semantics for LLVM IR. When certain instructions or semantic cases are not fully implemented, the engine conservatively falls back to fresh symbolic values. This preserves robustness but can reduce precision and classify values more conservatively. A fourth limitation is that interprocedural support remains intentionally narrow. The current direct-call handling is sufficient for straight-line callees, but not for arbitrary function boundaries, recursive call graphs, or richer modular summaries. Finally, the tool classifies values rather than proves whole-program security properties. Its output is an analysis artifact that can support downstream defenses and is not a standalone proof.

7.2 Next Steps

A next step is to strengthen loop reasoning, replacing the current first-iteration heuristic with inductive invariants or proof-oriented summaries for repeated control-flow structure. The benchmark suite can also be expanded. A larger corpus of cryptographic and security-critical programs would provide stronger empirical basis for judging precision, scalability, and robustness. Evaluating on more real-world kernels would help determine which instruction classes, memory behaviors, and symbolic patterns require improved support. Solver scalability can also be improved. Results show that

semantic complexity, not just path count, plays a substantial role in cost. Future work should investigate formula simplification, caching, path merging where sound, targeted timeouts, and principled query scheduling to make analysis more practical on larger, arithmetic-heavy kernels.

Finally, future work includes integration with complementary toolchains. The publicness judgments produced by the analysis are naturally suited to compiler-assisted selective mitigation. They can drive automatic insertion/removal CT protections, guide hardware/software cooperation mechanisms, or feed into broader speculative-security toolchains, making defenses less conservative and costly. This project is thus both a working prototype and a foundation for future work on LLVM-level, value-granular security analysis.

References

- [1] Frances E. Allen. 1970. Control flow analysis. *SIGPLAN Not.* 5, 7 (July 1970), 1–19. doi:10.1145/390013.808479
- [2] Manuel Barbosa, Gilles Barthe, Karthik Bhargavan, Bruno Blanchet, Cas Cremers, Kevin Liao, and Bryan Parno. 2021. SoK: Computer-aided cryptography. In *2021 IEEE symposium on security and privacy (SP)*. IEEE, 777–795.
- [3] Daniel J Bernstein. 2005. Cache-timing attacks on AES. (2005).
- [4] Rutvik Choudhary, Alan Wang, Zirui Neil Zhao, Adam Morrison, and Christopher W Fletcher. 2023. Declassiflow: A static analysis for modeling non-speculative knowledge to relax speculative execution security measures. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 2053–2067.
- [5] Rutvik Choudhary, Jiyong Yu, Christopher Fletcher, and Adam Morrison. 2021. Speculative privacy tracking (SPT): Leaking information from speculative execution without compromising privacy. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 607–622.
- [6] Sushant Dinesh, Grant Garrett-Grossman, and Christopher W Fletcher. 2022. Synthct: Towards portable constant-time code. In *NDSS*.
- [7] Marco Guarnieri, Boris Köpf, José F Morales, Jan Reineke, and Andrés Sánchez. 2020. Spectector: Principled detection of speculative information flows. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1–19.
- [8] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, et al. 2020. Spectre attacks: Exploiting speculative execution. *Commun. ACM* 63, 7 (2020), 93–101.
- [9] Paul C Kocher. 1996. Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems. In *Annual international cryptography conference*. Springer, 104–113.
- [10] Xiaoxuan Lou, Tianwei Zhang, Jun Jiang, and Yinqian Zhang. 2021. A survey of microarchitectural side-channel vulnerabilities, attacks, and defenses in cryptography. *ACM Computing Surveys (CSUR)* 54, 6 (2021), 1–37.
- [11] Nicholas Mosier, Hamed Nemati, John C Mitchell, and Caroline Trippel. [n. d.]. Protean: A Programmable Spectre Defense. ([n. d.]).
- [12] Nicholas Mosier, Hamed Nemati, John C Mitchell, and Caroline Trippel. 2024. Serberus: Protecting cryptographic code from spectres at compile-time. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 4200–4219.
- [13] Ofir Weisse, Ian Neal, Kevin Loughlin, Thomas F Wenisch, and Baris Kasikci. 2019. NDA: Preventing speculative execution attacks at their source. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 572–586.
- [14] Jiyong Yu, Lucas Hsiung, Mohamad El Hajj, and Christopher W Fletcher. 2018. Data oblivious ISA extensions for side channel-resistant and high performance computing. *Cryptology ePrint Archive* (2018).
- [15] Jiyong Yu, Namrata Mantri, Josep Torrellas, Adam Morrison, and Christopher W Fletcher. 2020. Speculative data-oblivious execution: Mobilizing safe prediction for safe and efficient speculative execution. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 707–720.
- [16] Zhiyuan Zhang, Gilles Barthe, Chitchanok Chuengsatiansup, Peter Schwabe, and Yuval Yarom. 2023. Ultimate {SLH}: Taking speculative load hardening to the next level. In *32nd USENIX Security Symposium (USENIX Security 23)*. 7125–7142.